MOBILE APP DEVELOPMENT 700

RICHFIELD
richfield.ac.za

Name and Surname: Tshiamo Peta

Student ITS No: 402411999

Qualification: Bsc IT Year of Study: 3rd Semester: 2nd

Assignment due date: 23/10/2026 Date submitted: 22/10/2026

QUESTION	EXAMINER MARKS	MODERATOR MARKS	REMARKS

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.

- ☐ Use Times New Roman font, size 12, with 1.5 line spacing throughout the document.
- ☐ Apply Harvard Referencing Style for all citations and references.

For essay-style assignments, please include the following sections:

- ☐ Table of Contents
- ☐ Introduction
- ☐ Main Body (with relevant subheadings)
- ☐ Conclusion
- ☐ References

- ☐ Submit the assignment in PDF format on Moodle.
- ☐ Use the specified cover page provided.
- ☐ Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:
I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism.

Student Signature: Tshiamo Peta Digitally signed by Tshiamo Peta Date: 2026.10.22 12:38:03 +02'00' Date: 22/10/2026

Richfield Graduate Institute of Technology (Pty) Ltd is registered with the Department of Higher Education & Training as a Private Higher Education Institution under the Higher Education Act, 1997, Registration Certificate No. 2000/HE07/008.

Declaration of Originality

I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgement is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism.

I further declare that I am able to explain and defend any part of the submitted code if asked to do so.

Table of Contents

Declaration of Originality 2

1. Introduction 4

2. System Design 5

   2.1 Architectural overview 5

   2.2 Screen flow 5

   2.3 Data model 6

3. The Strict-Matching Algorithm 7

   3.1 The rule 7

   3.2 Evaluation pipeline 8

   3.3 Worked example 8

   3.4 Optional extension 9

4. Screenshots of Every Output 10

5. Key Code Snippets 16

   5.1 Main Java 16

   5.2 Schema creation and one-time seeding 19

   5.3 Driving the RecyclerView 20

   5.4 Validation before writing 20

   5.5 Reloading in onResume 21

6. Challenges and Solutions 22

   6.1 String matching failed 22

   6.2 The list went stale after editing 22

   6.3 Guaranteeing the recipes existed before the first match 22

7. Conclusion and Reflection 23

8. References 24

Declaration of AI usage 25

1. Introduction

This assignment forms part of the practical assessment for Mobile App Development 700. Building on the module content covering Android Studio, the Activity lifecycle, layouts, intents, adapters, and persistent data storage, you are required to design, build, document, and demonstrate a fully functional Android application written in Java.

The Smart Pantry Manager helps a user reduce food waste by tracking the ingredients they actually have at home (their "pantry") and suggesting recipes they can cook using strictly those leftover ingredients - no shopping trip required, and no recipe suggested unless the user genuinely already has everything it needs.

The Smart Pantry Manager addresses that decision directly. It is a single-scope Android application that records the ingredients a user actually has at home and then suggests recipes that can be cooked with exactly those ingredients. Its defining feature is a strict matching rule: a recipe is only ever suggested when every single ingredient it requires is already in the pantry, in at least the required quantity.

2. System Design

2.1 Architectural overview

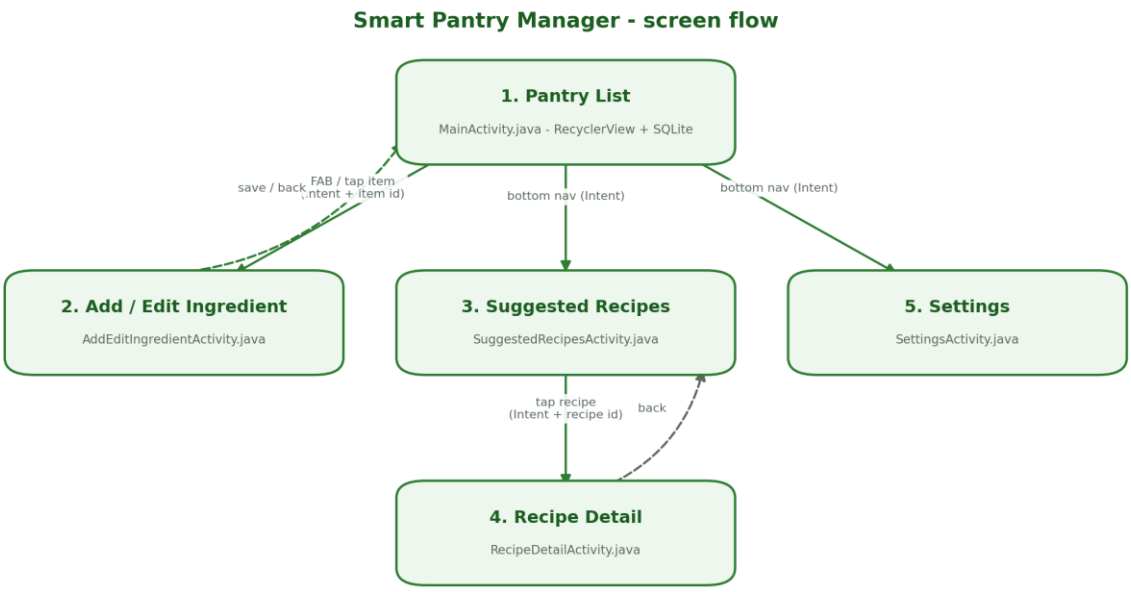
The application is divided into three layers. Each layer has one responsibility, and the dependencies only ever point downwards:

- **Data layer (data).** SQLiteOpenHelper owns the schema, the CRUD operations and the one-time seeding of the recipe collection. No activity writes raw SQL.
- **Logic layer (logic).** RecipeMatcher, IngredientNameNormalizer and UnitConverter contain the business rule and contain no Android imports at all. Because they are plain Java, they can be executed by JUnit on the development machine in milliseconds, without an emulator.
- **Presentation layer (ui, model, util).** Five Activities, two RecyclerView adapters, the XML layouts and a small SharedPreferences wrapper. Activities only translate user actions into calls on the lower layers.

Separating the matching rule from the screens was the most important design decision in the project. It means the rule that carries the marks cannot be weakened accidentally by a screen, and it can be proven correct with automated tests.

2.2 Screen flow

The application consists of five distinct Activities. The pantry list is the entry point and hosts the bottom navigation bar that reaches the suggested recipes and settings screens. Figure 2.1 shows the navigation graph.



Five screens, each implemented as its own Activity. Data is passed between screens with Intents; the pantry is read from SQLite.

Table 2.1 lists every transition and the data that travels with it. Each row corresponds to an explicit Intent, satisfying the requirement for correct use of Intents to navigate between screens and pass data.

Table 2.1: Intent transitions between the five screens.

2.3 Data model

The application uses a local SQLite database named smartpantry.db, version 1, with three tables. The pantry is owned by the user and changes constantly; the recipe collection is static seed data. Figure 2.2 shows the entity relationship diagram.

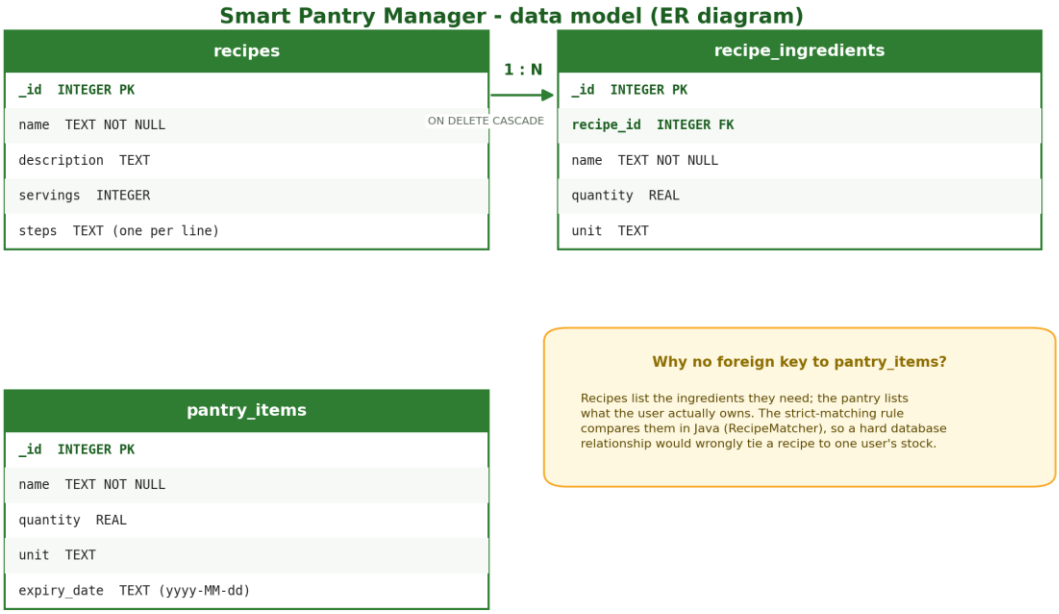


Table 2.2: Database tables.

There is deliberately no foreign key between pantry_items and recipes. A recipe describes what is needed; the pantry describes what is owned. The relationship between the two is a comparison performed at runtime by the matching algorithm, not a structural relationship in the schema. Adding a foreign key would wrongly imply that a recipe belongs to one user's stock, and it would make the seed data unusable as soon as the user changed a quantity.

The recipe collection is seeded inside SQLiteOpenHelper.onCreate(), which the framework calls exactly once, the first time the database is created. Seeding inside a transaction guarantees that the collection is either completely present or completely absent; a failure halfway through cannot leave a partial collection behind. The recipe_ingredients rows use ON DELETE CASCADE so deleting a recipe in a future version also removes its ingredient rows automatically.

3. The Strict-Matching Algorithm

3.1 The rule

A recipe may only be shown as suggested if every ingredient it requires is currently present in the user's pantry, in at least the required quantity. If a recipe needs five ingredients and the pantry contains four, the recipe is excluded — not shown with a warning, not shown greyed out, excluded.

The rule is implemented in RecipeMatcher.checkIngredient(), which is called once per required ingredient. Each recipe is therefore reduced to a list of IngredientCheck results, and MatchResult.isFullMatch() returns true only when the list of missing ingredients is empty. That single condition is the entire "strict" requirement, which makes it easy to demonstrate and easy to test.

3.2 Evaluation pipeline

1. **Normalise the name.** the name is lower-cased, punctuation and bracketed notes are removed, preparation and quantity words are dropped, plurals are converted to singular and a short synonym list is applied.
2. **Reject incomparable units.** mass, volume and countable units are different families. Grams can be compared with kilograms, but grams can never be compared with pieces. This prevents a false match between 200 g of cheese and two pieces of cheese.
3. **Convert and accumulate.** every pantry entry with the same normalised name and a compatible unit is converted to the family's base unit (g, ml or pieces) and summed, so 0.5 kg of butter plus 60 g of butter counts as 560 g.
4. **Compare.** the ingredient is satisfied only when the accumulated quantity is at least the required quantity, with a small epsilon to absorb floating-point error.

3.3 Worked example

The shipped demo pantry contains fourteen ingredients: six eggs, eight slices of bread, 200 g of cheese, three tomatoes, two onions, four cloves of garlic, 500 g of rice, 500 g of pasta, 1 L of milk, 250 g of butter, 1 kg of flour, 750 ml of cooking oil, 500 g of salt and 1 kg of sugar. Against the seeded collection of twenty recipes this produces eight recipes in the strict list (Cheese Omelette, Scrambled Eggs on Toast, Grilled Cheese Sandwich, Garlic Butter Pasta, White Sauce Pasta, Mac and Cheese, Cheesy Garlic Bread and Cheese and Tomato Toastie) and four in the optional "Almost there" list.

Deleting the single cheese row demonstrates the rule precisely. Six recipes lose their cheese requirement and immediately leave the strict list, leaving only Scrambled Eggs on Toast and Garlic Butter Pasta cookable, while the "Almost there" list grows from four to ten because those six recipes are now exactly one ingredient short. The pantry itself shrinks by one row. This behaviour is verified automatically by the unit test

SeedDataTest.removingCheeseRemovesCheesyRecipesFromTheSuggestions, so a future edit to the recipe data that breaks the demonstration fails the build rather than the video.

3.4 Optional extension

The brief permits a bonus “Almost There” list provided it is clearly separated from the strict suggestions. RecipeMatcher.almostMatches() returns recipes whose missing list contains exactly one entry. The suggestions screen renders the two groups under separate headers, and the extension can be switched off entirely from the settings screen, so the strict list is never ambiguous.

4. Screenshots of Every Output

Every screenshot below was captured from the running application on an Android device. Each is followed by a caption explaining what it demonstrates.



Figure 4.1: Empty pantry state.



Figure 4.2: The recipe list, still empty

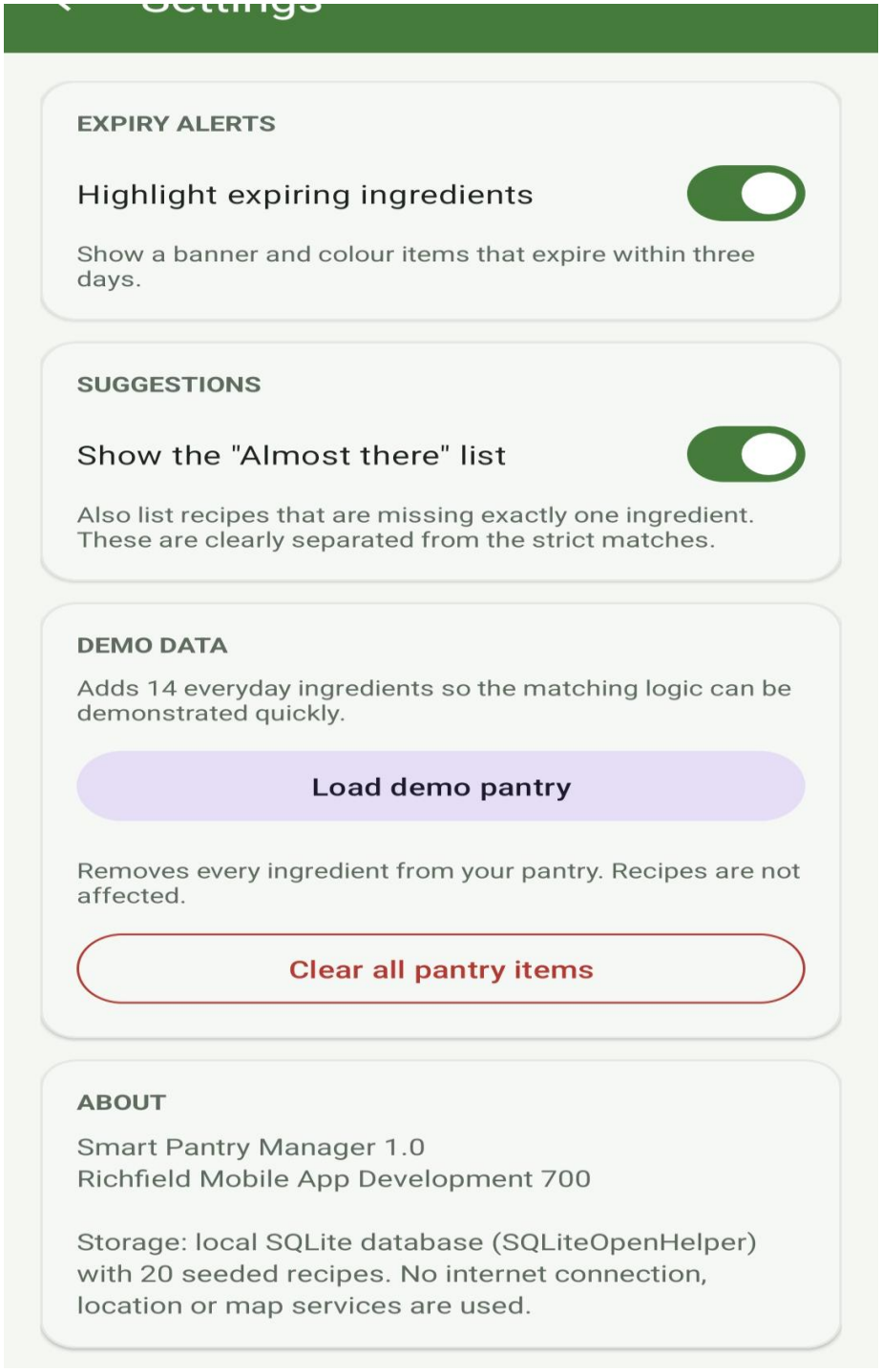


Figure 4.3: Settings Page

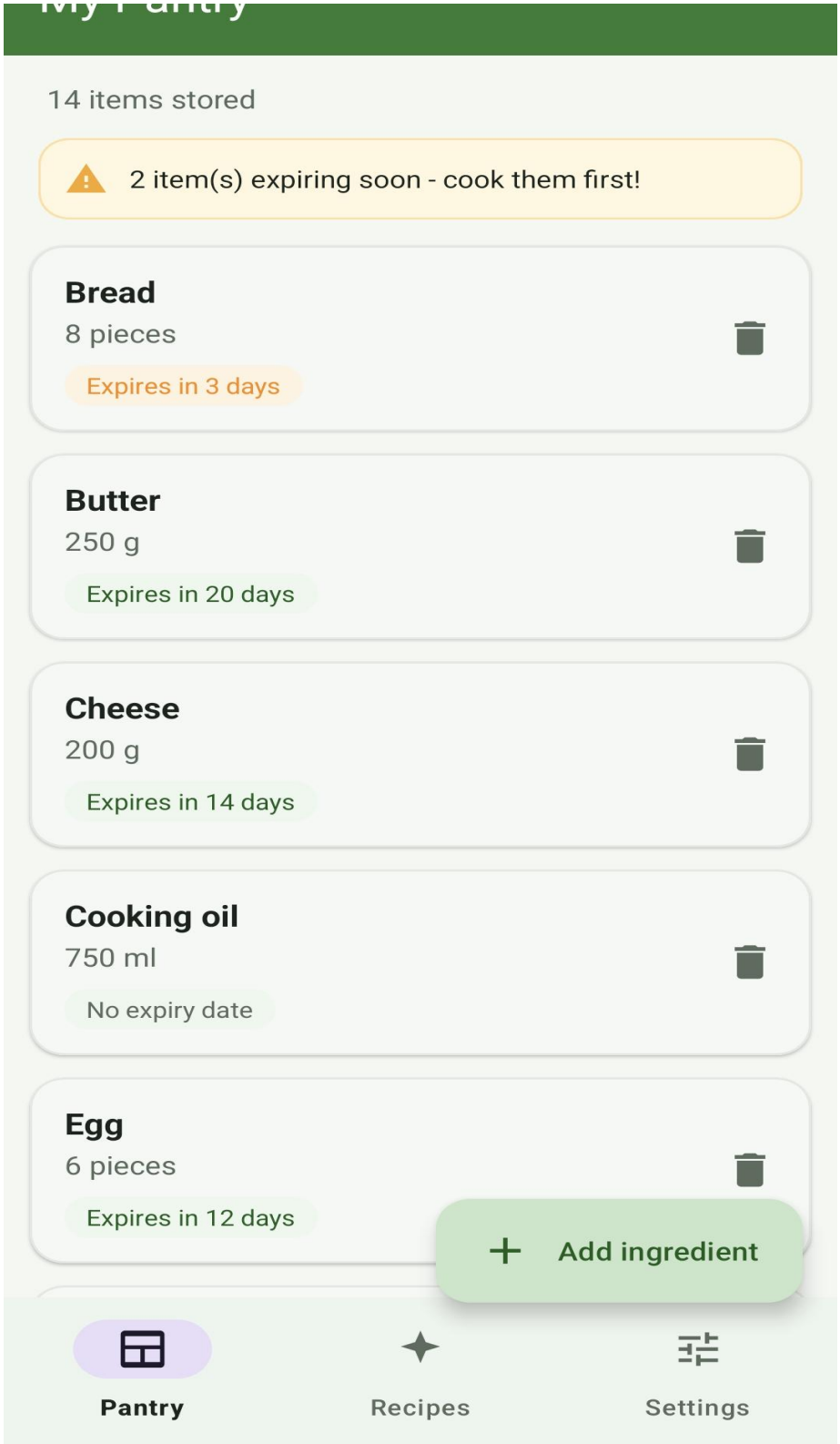


Figure 4.4: Pantry with the demo ingredients added

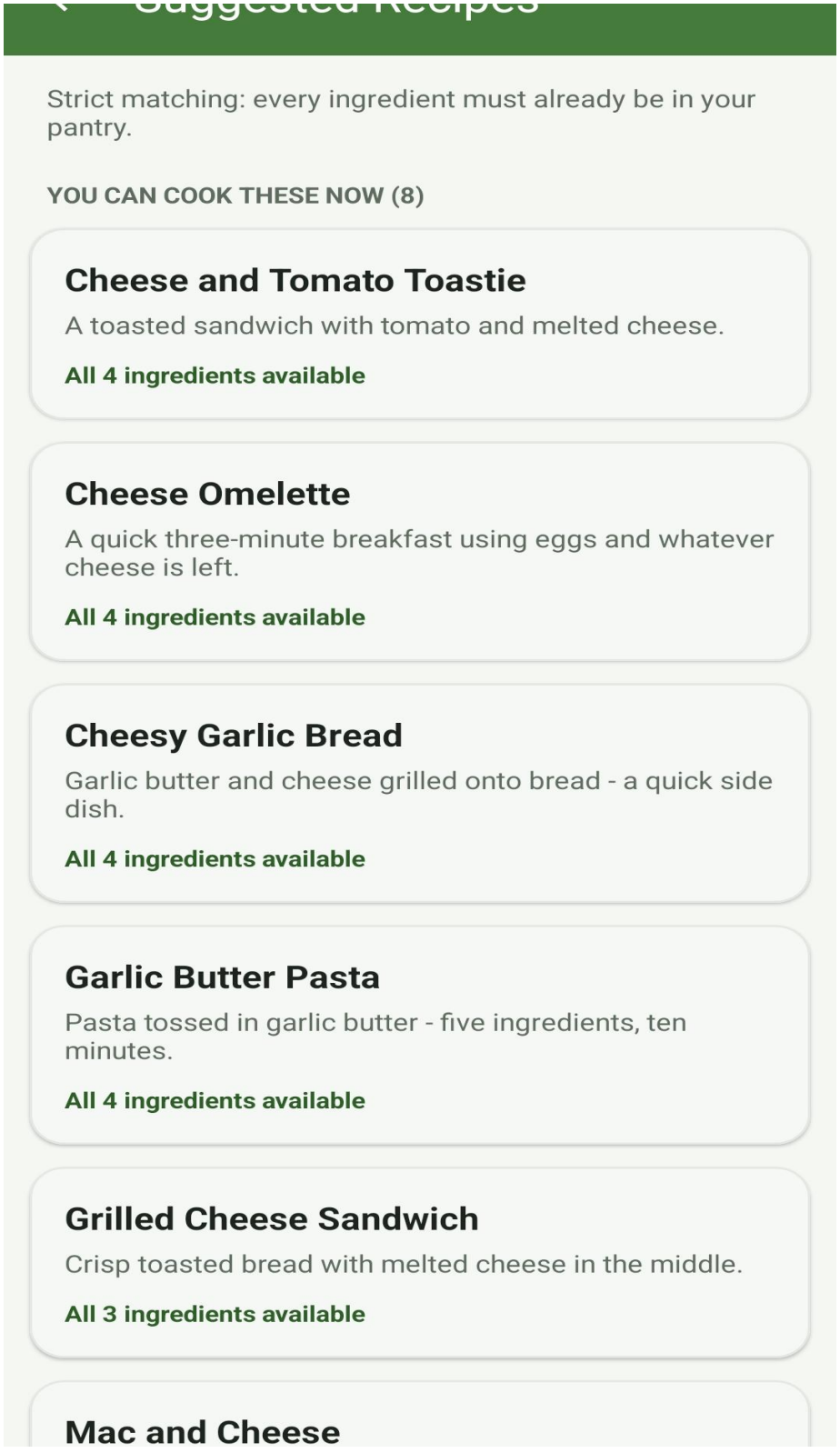


Figure 4.5: Suggested Recipes from what is in stock on the pantry

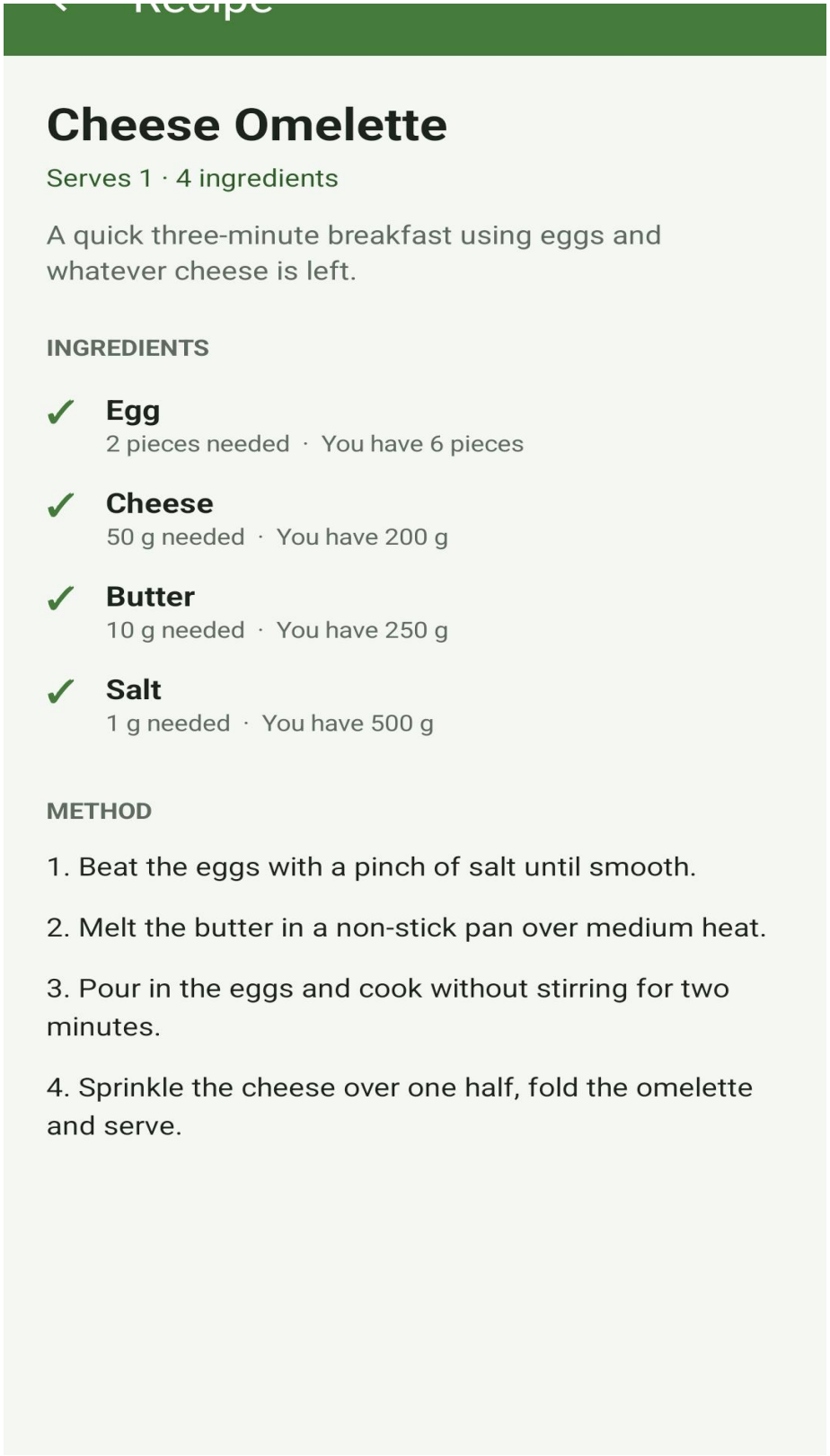


Figure 4.6: Recipe with all the ingredients listed

5. Key Code Snippets

Full code can be found in the Github repo

5.1 Main Java

```
package za.co.richfield.smartpantry;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.LinearLayoutManager;

import com.google.android.material.dialog.MaterialAlertDialogBuilder;

import java.util.List;

import za.co.richfield.smartpantry.data.DatabaseHelper;
import za.co.richfield.smartpantry.databinding.ActivityMainBinding;
import za.co.richfield.smartpantry.model.PantryItem;
import za.co.richfield.smartpantry.ui.PantryAdapter;
import za.co.richfield.smartpantry.util.AppPreferences;
import za.co.richfield.smartpantry.util.DateUtils;

// Pantry List
public class MainActivity extends AppCompatActivity implements
PantryAdapter.OnPantryItemListener {

    private static final int EXPIRING_SOON_DAYS = 3;

    private ActivityMainBinding binding;
    private DatabaseHelper databaseHelper;
    private AppPreferences preferences;
    private PantryAdapter adapter;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        binding = ActivityMainBinding.inflate(getLayoutInflater());
        setContentView(binding.getRoot());

        databaseHelper = DatabaseHelper.getInstance(this);
```

```
        preferences = new AppPreferences(this);

        adapter = new PantryAdapter(this);
        binding.recyclerPantry.setLayoutManager(new LinearLayoutManager(this));
        binding.recyclerPantry.setAdapter(adapter);

        binding.fabAddItem.setOnClickListener(v -> openAddIngredientScreen());
        binding.buttonAddFirstItem.setOnClickListener(v ->
openAddIngredientScreen());

        binding.bottomNavigation.setOnItemSelectedListener(item -> {
            int itemId = item.getItemId();
            if (itemId == R.id.nav_suggested) {
                startActivity(new Intent(MainActivity.this,
SuggestedRecipesActivity.class));
                return true;
            }
            if (itemId == R.id.nav_settings) {
                startActivity(new Intent(MainActivity.this,
SettingsActivity.class));
                return true;
            }
            return itemId == R.id.nav_pantry;
        });
        binding.bottomNavigation.setSelectedItemId(R.id.nav_pantry);
    }

    @Override
    protected void onResume() {
        super.onResume();
        // If the user returns from another tab, put the selection back on
"Pantry".
        binding.bottomNavigation.setSelectedItemId(R.id.nav_pantry);
        loadPantry();
    }

    // Pantry is saved in SQlite
    private void loadPantry() {
        List<PantryItem> items = databaseHelper.getAllPantryItems();

        adapter.setHighlightExpiry(preferences.isExpiryAlertsEnabled());
        adapter.submitItems(items);

        binding.textPantryCount.setText(getString(R.string.pantry_subtitle,
items.size()));
```

```
        boolean isEmpty = items.isEmpty();
        binding.layoutEmptyState.setVisibility(isEmpty ? View.VISIBLE : View.GONE);
        binding.recyclerPantry.setVisibility(isEmpty ? View.GONE : View.VISIBLE);
        binding.fabAddItem.setVisibility(isEmpty ? View.GONE : View.VISIBLE);

        updateExpiryBanner(items);
    }

    //Checks items that are expiring and shows a warning to those near expiry date
    private void updateExpiryBanner(List<PantryItem> items) {
        if (!preferences.isExpiryAlertsEnabled()) {
            binding.cardExpiryBanner.setVisibility(View.GONE);
            return;
        }
        int expiring = 0;
        for (PantryItem item : items) {
            if (!item.hasExpiryDate()) {
                continue;
            }
            if (DateUtils.isExpired(item.getExpiryDate())
                    || DateUtils.isExpiringSoon(item.getExpiryDate(),
EXPIRING_SOON_DAYS)) {
                expiring++;
            }
        }
        if (expiring > 0) {
            binding.cardExpiryBanner.setVisibility(View.VISIBLE);
            binding.textExpiryBanner.setText(getString(R.string.expiring_banner,
expiring));
        } else {
            binding.cardExpiryBanner.setVisibility(View.GONE);
        }
    }

    private void openAddIngredientScreen() {
        startActivity(AddEditIngredientActivity.createIntent(this,
AddEditIngredientActivity.NO_ID));
    }

    @Override
    public void onEditItem(PantryItem item) {
        startActivity(AddEditIngredientActivity.createIntent(this, item.getId()));
    }
```

```
    @Override
    public void onDeleteItem(final PantryItem item) {
        new MaterialAlertDialogBuilder(this)
                .setTitle(R.string.delete_item_title)
                .setMessage(getString(R.string.delete_item_message,
item.getName()))
                .setNegativeButton(R.string.cancel, null)
                .setPositiveButton(R.string.delete, (dialog, which) -> {
                    databaseHelper.deletePantryItem(item.getId());
                    Toast.makeText(this, getString(R.string.item_deleted,
item.getName()),
                            Toast.LENGTH_SHORT).show();
                    loadPantry();
                })
                .show();
    }
}
```

5.2 Schema creation and one-time seeding

```
// data/DatabaseHelper.java
@Override
public void onCreate(SQLiteDatabase db) {
    db.execSQL(CREATE_TABLE_PANTRY);
    db.execSQL(CREATE_TABLE_RECIPES);
    db.execSQL(CREATE_TABLE_RECIPE_INGREDIENTS);
    seedRecipes(db);
}

private void seedRecipes(SQLiteDatabase db) {
    List<Recipe> recipes = DatabaseSeeder.buildSeedRecipes();
    db.beginTransaction();
    try {
        for (Recipe recipe : recipes) {
            insertRecipe(db, recipe);
        }
        db.setTransactionSuccessful();
    } finally {
        db.endTransaction();
    }
}
```

5.3 Driving the RecyclerView

```
// ui/PantryAdapter.java
public void submitItems(List<PantryItem> newItems) {
    items.clear();
    if (newItems != null) {
        items.addAll(newItems);
    }
    notifyDataSetChanged();
}

void bind(final PantryItem item) {
    binding.textName.setText(item.getName());
    double baseQuantity = UnitConverter.toBaseUnit(item.getQuantity(),
item.getUnit());
    binding.textQuantity.setText(UnitConverter.formatQuantity(baseQuantity,
item.getUnit()));
    binding.cardRoot.setOnClickListener(v -> listener.onEditItem(item));
    binding.buttonDelete.setOnClickListener(v -> listener.onDeleteItem(item));
}
```

5.4 Validation before writing

```
// AddEditIngredientActivity.java
String name = textOf(binding.inputName).trim();
if (TextUtils.isEmpty(name)) {
    binding.inputLayoutName.setError(getString(R.string.error_name_required));
    valid = false;
} else if (name.length() < 2) {
    binding.inputLayoutName.setError(getString(R.string.error_name_too_short));
    valid = false;
}

try {
    quantity = Double.parseDouble(quantityText.replace(',', '.'));
    if (quantity <= 0) {

binding.inputLayoutQuantity.setError(getString(R.string.error_quantity_invalid));
        valid = false;
    }
} catch (NumberFormatException e) {

binding.inputLayoutQuantity.setError(getString(R.string.error_quantity_invalid));
    valid = false;
}
```

5.5 Reloading in onResume

```
// MainActivity.java
@Override
protected void onResume() {
    super.onResume();
    binding.bottomNavigation.setSelectedItemId(R.id.nav_pantry);
    loadPantry();   // re-reads SQLite every time the screen becomes visible
}

private void loadPantry() {
    List<PantryItem> items = databaseHelper.getAllPantryItems();
    adapter.setHighlightExpiry(preferences.isExpiryAlertsEnabled());
    adapter.submitItems(items);
    binding.textPantryCount.setText(getString(R.string.pantry_subtitle,
items.size()));
    binding.layoutEmptyState.setVisibility(items.isEmpty() ? View.VISIBLE :
View.GONE);
    updateExpiryBanner(items);
}
```

6. Challenges and Solutions

6.1 String matching failed

The first version on the matcher compared the ingredient names with equals(). It worked during seed data but failed the moment a pantry rowed like “Apples” was not equated to “apple”. Problems also arose with measurements like 500g not being equated to 0.5kg. That’s where the Ingredient Name Normalizer came in to normalize the size of what was being compared so it could match. UnitConverter also served the same purpose but with numbers so quantities had the same base unit of measurement

6.2 The list went stale after editing

After adding an ingredient the pantry list did not show it until the app was restarted. My first attempt passed results back between activities, which worked for one path but broke for settings, where the pantry can be cleared or replaced entirely. As soon as I had three screens that could change the same table, that approach became unmaintainable. The solution was to treat SQLite as the single source of truth and reload in onResume() on every screen that displays data. It costs one small query per screen appearance and removed an entire class of bug.

6.3 Guaranteeing the recipes existed before the first match

Seeding the twenty recipes from the first activity worked in testing and was fragile by design: two entry points into the app could seed twice, and a rotation in the middle of seeding could produce a partial collection. Moving the seed into SQLiteOpenHelper.onCreate(), which the framework guarantees runs once, and wrapping the inserts in a transaction, made the problem disappear.

7. Conclusion and Reflection

The Smart Pantry Manager satisfies the specification: a Java Android app with 5 screens, CRUD operations for a local SQLite database, 2 custom RecyclerView adapters, input validation, activity switching using Intents, and a bottom navigation bar, which uses the strict-matching rule (identified in the assignment as the most important piece of logic). The rule and the normalizer, unit converter and shipped seed data are tested via automated tests.

The most valuable lesson was about where to put logic. My instinct was to write the matching inside the suggestions activity, where it is used. Extracting it into a plain Java class took an hour and repaid that hour many times over: it made the rule testable without an emulator, it made the rule explainable in one screen of code, and it made it impossible for a later UI change to loosen it quietly. The second lesson was that on Android, the data layer is the state. Once every screen reloaded from SQLite in onResume(), persistence and cross-screen consistency stopped being features I had to implement and became consequences of the architecture.

If I had better time management and the scope of the project allowed I would add a shopping list and a connector to apps like Pick n Pay Asap or Chekers Sixty60 to be it when you add items to your “Shopping Cart” on The my pantry app they get added to you checkout bag in external apps.

8. References

Malone, D. (2024, September 21). *DJ Malone*. Retrieved from Youtube.com: https://tinyurl.com/uanpt2jm

Steps, F. (2025, April 30). *Few Steps*. Retrieved from Youtube.com: https://tinyurl.com/y4y26vs7

Android Developers (2025) Create dynamic lists with RecyclerView. Available at: https://developer.android.com/develop/ui/views/layout/recyclerview (Accessed: 21 October 2026).

Android Developers (2025) Meet Android Studio. Available at: https://developer.android.com/studio/intro (Accessed: 21 October 2026).

Android Developers (2025) Save data using SQLite. Available at: https://developer.android.com/training/data-storage/sqlite (Accessed: 21 October 2026).

Android Developers (2025) SQLiteOpenHelper. Available at: https://developer.android.com/reference/android/database/sqlite/SQLiteOpenHelper (Accessed: 21 October 2026).

Android Developers (2025) Common intents and intent filters. Available at: https://developer.android.com/guide/components/intents-filters.

Food and Agriculture Organization of the United Nations (2011) Global Food Losses and Food Waste: Extent, Causes and Prevention. Rome: FAO.

Google (2025) Material Components for Android. Available at: https://github.com/material-components/material-components-android (Accessed: October 2026).

JUnit Team (2024) JUnit 4. Available at: https://junit.org/junit4/.

Declaration of AI usage

AI was used to format the overall project structure, i.e the layout, and was also used for error handling both on the part of the code and missing dependencies needed to operate Android studio on a personal device.

AI was also used to generate data for the SQLite database and the screen flow